



A RAG-Based Chatbot for Slovenian Tax and Investment Law

Maj Bregar, Jan Tršan, and Edita Džubur

Abstract

This project develops a domain-specific chatbot for Slovenian investment and tax law. Since general-purpose LLMs lack reliable domain knowledge, the proposed approach uses retrieval-augmented generation (RAG) to produce accurate, up-to-date, and verifiable responses based on external legal documents. Prompt engineering is applied to enforce structure and introduce guardrails. The system is built on a curated dataset combining legal corpora and official Slovenian sources.

Keywords

LLM, retrieval-augmented generation, chatbot, prompt engineering

Advisors: Slavko Žitnik

Introduction

Large language models (LLMs) often lack sufficiently deep domain-specific knowledge to provide reliable advice in specialized fields. This is especially problematic in legal and financial domains, where answers may depend on specific provisions, exceptions, and recent changes. In such settings, responses should not only be plausible, but also grounded in sources that the user can inspect.

This project develops a chatbot for Slovenian investment and tax law. Instead of relying only on the model's internal knowledge, the system uses retrieval-augmented generation (RAG) to ground answers in external legal documents. The goal is to provide responses that are understandable, relevant to the user's question, and linked to sources that can be verified.

Literature Overview

Several approaches can be used to adapt LLMs to specialized tasks. They differ mainly in whether knowledge is provided through the prompt, learned into the model parameters, or retrieved from external sources at inference time.

A straightforward and widely adopted approach is prompt engineering [1], where model behavior is guided through carefully designed instructions, constraints, or examples. Rather than modifying the model itself, knowledge is implicitly introduced at inference time through the prompt. This makes the approach highly flexible and inexpensive, and particularly useful for rapid prototyping, enforcing output structure, or guiding reasoning processes in chatbot systems. However,

its reliance on the context window limits how much domain knowledge can be injected, and performance can vary significantly depending on prompt formulation. As a result, prompt engineering alone is often insufficient in domains where correctness and reliability are critical.

In contrast, fine-tuning [2] embeds domain knowledge directly into the model's parameters by training on domain-specific data. This allows the model to internalize terminology, patterns, and expected behaviors, leading to more consistent and specialized outputs. Such properties are beneficial for tasks like document classification or maintaining a specific response style in a chatbot. At the same time, this approach comes with notable drawbacks: it is resource-intensive, depends on the availability of high-quality labeled data, and produces a static representation of knowledge that cannot easily adapt to updates, such as changes in regulations or financial markets. Furthermore, the lack of explicit source attribution limits its applicability in settings where transparency is required.

RAG [3] addresses many of these limitations by combining pretrained model knowledge with external information sources. Instead of relying solely on internal parameters, the model retrieves relevant documents at inference time and conditions its responses on them. This enables access to up-to-date and domain-specific information, reduces hallucinations when retrieval is effective, and supports the inclusion of explicit references. Consequently, RAG is particularly well-suited for legal and financial chatbot systems, where answers must be both accurate and verifiable. The trade-off lies in increased system complexity, as it requires a dedicated retrieval

pipeline, and in sensitivity to retrieval quality, which directly impacts response reliability and latency.

In practice, these methods are rarely used in isolation. Prompt engineering is often employed to structure and constrain outputs, fine-tuning to ensure consistent behavior, and RAG to provide grounded, up-to-date knowledge, resulting in hybrid systems that balance flexibility, accuracy, and transparency.

Proposed Dataset and Data Sources

The dataset combines Slovenian legal texts with explanatory government sources and linguistic corpora. The main focus is on tax- and investment-related legislation, especially acts such as *ZDoh-2* and *ZDavP-2*.

Core Legal Corpus

The primary dataset is based on the Slovenian legal corpus provided within the LLM4DH project [4]. It contains Slovenian legislation in a structured form and provides the core legal material used for retrieval and grounding.

Supplementary Legal Data Sources

To improve coverage beyond the core corpus, we also use official Slovenian legal and administrative sources. **PISRS (Pravno-informacijski sistem RS)**¹ provides structured access to Slovenian legislation, including tax-related acts, while **Uradni list RS**² contains official publications of laws and amendments.

For more practical user-facing material, we also include **FURS (Financial Administration of RS)**³, which provides tax guidelines, explanations, and frequently asked questions, and **e-Uprava**⁴, which offers simplified explanations of administrative procedures and legal obligations.

CLARIN.SI Legal and Linguistic Corpora

We also use resources from the CLARIN.SI infrastructure to support linguistic processing and possible future model evaluation. The **Slovenian Legal Corpus (CLARIN.SI)**⁵ contains Slovenian legal texts with linguistic annotations and metadata. The **siParl Parliamentary Corpus**⁶ contains Slovenian parliamentary debates from 1990 to 2018 and provides broader context for legislative discussions [5].

These resources are useful for terminology analysis, linguistic preprocessing, and potential later experiments beyond the initial RAG pipeline.

Dataset Construction Strategy

The dataset is constructed by combining authoritative Slovenian legal texts with supporting explanatory sources. The

documents are cleaned and converted into a consistent textual format before being passed to the chunking pipeline described below.

The focus is on tax- and investment-related legislation, while sources such as FURS and e-Uprava provide more practical explanations of legal procedures. This gives the system access to both formal legal provisions and user-oriented interpretations.

Methodology

This section describes the implemented baseline pipeline, from legal document preprocessing to retrieval, reranking, and answer generation.

System Architecture

The implemented baseline follows a modular RAG architecture with separate offline indexing and online answering stages. The offline stage builds the retrieval store from legal documents, while the online stage retrieves and reranks relevant chunks before generating a grounded answer with a locally served instruction-tuned language model.

The system is split into two main runtime components. The first runs the vLLM server and exposes an OpenAI-compatible endpoint for answer generation. The second acts as the chatbot client, loading the retrieval components, assembling prompts, managing chat history, and providing the interactive terminal interface. This separation is useful in the HPC environment because the LLM server can run as a GPU job, while retrieval, reranking, prompt construction, and chat-history handling remain on the client side.

Chunking Strategy

The retrieval component of the chatbot is built by converting legal documents into structured text chunks and embedding them into a vector index. The input consists of COLESLAW-style JSONL records, where each record contains metadata such as the law title, identifier, and full legal text.

The chunking process first filters the input records according to one of three strategies. In the default strategy, records are selected using a combination of explicitly whitelisted laws and keyword-based relevance filtering for tax and investment terminology. In the whitelist strategy, only a predefined set of core laws is retained, including *ZDoh-2*, *ZDavP-2*, *ZDDPO-2*, *ZDDV-1*, *ZTFI-1*, *ZISDU-3*, *ZBan-3*, and *ZGD-1*. In the all-laws strategy, filtering is disabled and all input records are processed; records that cannot be parsed into articles may optionally be preserved as fallback document-level chunks.

Each selected legal document is then segmented according to its internal legal structure. The parser detects article markers such as *1. člen*, extracts article headings when present, assigns chapter or section metadata, and splits article bodies into paragraph-level units such as *(1)*, *(2)*, and similar legal subdivisions. For long provisions, the parser applies additional splitting based on legal list boundaries, including

¹<https://pisrs.si>

²<https://www.uradni-list.si>

³<https://www.fu.gov.si>

⁴<https://e-uprava.gov.si>

⁵<https://www.clarin.si/repository/xmlui/handle/11356/2095>

⁶<https://www.clarin.si/repository/xmlui/handle/11356/1300>

numbered and lettered items. This avoids arbitrary cuts in the middle of words or legal clauses and helps preserve the semantic integrity of each chunk.

Each generated chunk contains both the raw chunk text and structured metadata. The metadata includes the law title, law code, article number, article heading, paragraph or item number, chapter, document identifiers, and domain tags such as `tax`, `income_tax`, `capital_gains`, `investment`, or `vat`. A formatted version of the chunk is also created, combining the legal text with its metadata. This formatted representation is used for embedding so that retrieval can benefit from both the content of the provision and its legal context.

This strategy is designed to balance retrieval precision with legal interpretability. Article and paragraph-level chunks are small enough to support focused retrieval, while the attached metadata preserves the legal source context needed for citation and verification.

Chunk Embedding

Chunk embeddings are generated using the multilingual sentence-transformer model `BAAI/bge-m3` [6], which provides strong multilingual performance and works well for morphologically rich languages such as Slovenian.

The resulting vectors are normalized by default and stored as a NumPy matrix. A FAISS index is then optionally constructed using inner product search over the normalized embeddings, which corresponds to cosine similarity. The final output directory contains the chunk file, embedding matrix, FAISS index.

RAG Pipeline

Upon receiving a user query, the text is embedded using `BAAI/bge-m3`, and the top 50 most similar chunks are retrieved from the RAG chunk store. These candidates are then reranked using `BAAI/bge-reranker-v2-m3` [7], with the top 10 most relevant chunks retained. The selected chunks, together with their metadata (e.g., law name, article, paragraph), are used to construct a structured RAG prompt that clearly delineates the legal context and provides explicit instructions for grounded answering.

The RAG prompt, along with the user query, system-level formatting instructions, and conversation history, is sent to a vLLM server running `Llama-3.1-8B-Instruct` [8] via the OpenAI API message interface. The prompt enforces strict reliance on the provided legal context, requiring the model to cite relevant laws and articles, avoid hallucinations, and explicitly indicate when the answer cannot be determined from the retrieved sources.

Evaluation Dataset and Preliminary Analysis

We prepared a gold evaluation dataset for Slovenian tax-related question answering. The dataset contains practical user questions based on several Slovenian tax laws, including scenario-based questions, edge cases, definition questions,

and a small number of unanswerable questions. The purpose of including different question types is to test not only whether the system can retrieve relevant legal provisions, but also whether it avoids generating unsupported answers when the available context is insufficient.

The evaluation examples were created with the help of several LLMs. We used them to generate possible tax-related questions, suggest answers, and check the answers against the relevant legal sources. This gave us a wider set of examples than we would likely get from writing all questions manually. After that, we manually reviewed the dataset and corrected the answers where needed. In particular, we checked whether each answer was supported by the cited legal text and whether the question represented a realistic use case for a Slovenian tax assistant.

We also ran the current chatbot pipeline on the gold questions and manually compared the generated answers with the reference answers. This preliminary analysis was qualitative rather than metric-based, but it helped reveal several recurring issues. In some cases, the system retrieved or relied on a less relevant legal act, including older sources. Some answers were partially correct but missed important exceptions or conditions, while others were overly cautious and stated that the answer could not be determined even when the gold answer was clear. We also observed a few cases where the model added unsupported details or misinterpreted legal negation.

Future Work

In future work, we plan to make the evaluation more systematic. For each question, we will store the retrieved chunks, the generated answer, and the sources used by the model, then score the result against the gold answer and expected legal sources. The evaluation should measure answer correctness, completeness, source relevance, and faithfulness to the provided legal context.

We also plan to combine manual review with LLM-based evaluation. A stronger LLM can help flag incomplete, unsupported, or contradictory answers, while manual inspection remains important for borderline legal cases. The results will then be used to tune retrieval and reranking parameters, filter outdated or less relevant sources, and improve the final prompting layer.

References

- [1] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Nee-lakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [2] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Liang Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3, 2022.

- [3] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- [4] LLM4DH Project. Llm4dh slovenian legal corpus, 2026. Internal dataset provided within the LLM4DH project.
- [5] CLARIN.SI. siparl: Slovene parliamentary corpus 1990–2018, 2019. Annotated corpus of Slovenian parliamentary proceedings.
- [6] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. Bge m3-embedding: Multilingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *arXiv preprint arXiv:2402.03216*, 4(5), 2024.
- [7] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muenighoff. C-pack: Packaged resources to advance general chinese embedding, 2023.
- [8] Meta Llama. Meta llama-3.1-8b-instruct, 2024.